

23CSE373 · COMPUTER VISION · GROUP 10

Fast Object Proposal Generation Using Handcrafted Gradient Features

*A from-scratch re-implementation and study of BING —
Binarized Normed Gradients for Objectness Estimation
(Cheng, Zhang, Lin, Torr · CVPR 2014)*

Abhinav Dileep AM.SC.U4CSE23202

Ayyappadas M T AM.SC.U4CSE23209

Hari Sankar A AM.SC.U4CSE23221

Parthiv M AM.SC.U4CSE23240

Sai Kishen K M AM.SC.U4CSE23248



Why object proposals?

Detection is expensive. A sliding-window detector must classify every position $\times$ scale $\times$ aspect ratio — easily 10^5 – 10^6 windows per image.

Most windows are background. Only a handful of windows actually contain an object.

Objectness first. A cheap, class-agnostic score ranks windows so that a detector only examines the top ~ 1000 .

Question we answer: “Which regions of this image are likely to contain objects?” — not “what is the object?”

$\sim 10^5 +$

windows a naïve sliding-window scan evaluates per image

95.1%

of objects recovered by our BING in the top 1000 proposals (synthetic test set)

188 fps

fastest configuration of our Python/Numba implementation

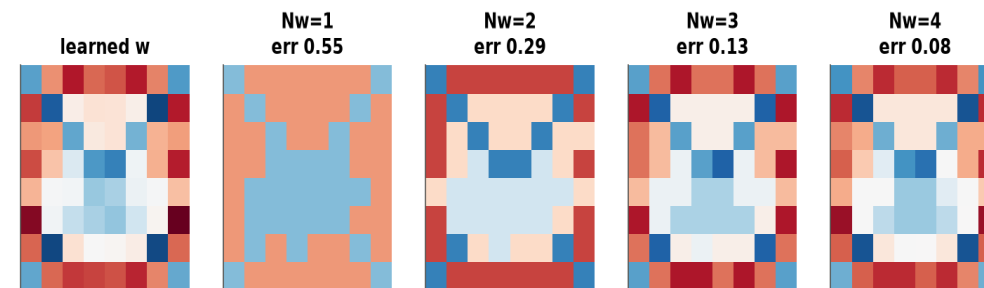
BING in one idea

Objects are stand-alone things with closed boundaries. Resize any window to a tiny 8×8 and look at its gradient magnitudes: object windows show a ring of strong edges around a quieter centre.

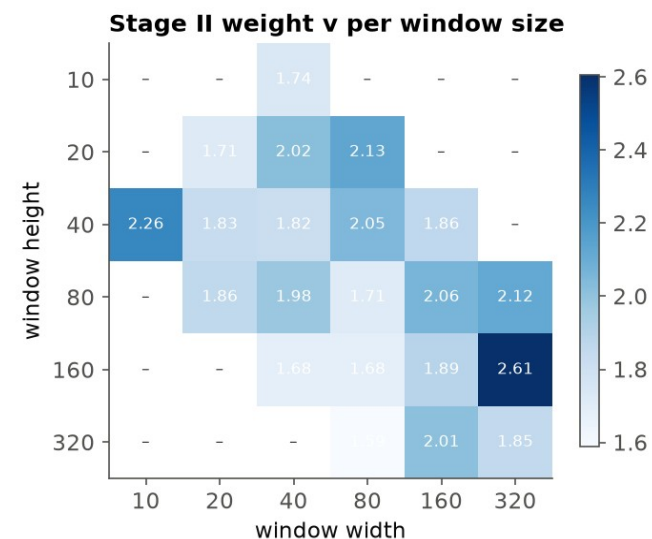
Normed gradients (NG). $g = \min(|g_x| + |g_y|, 255)$ — a 64-D feature that is invariant to translation, scale and aspect ratio of the window.

Linear model, two stages. An SVM filter w scores every window; a second per-size SVM calibrates scores across the 36 window sizes.

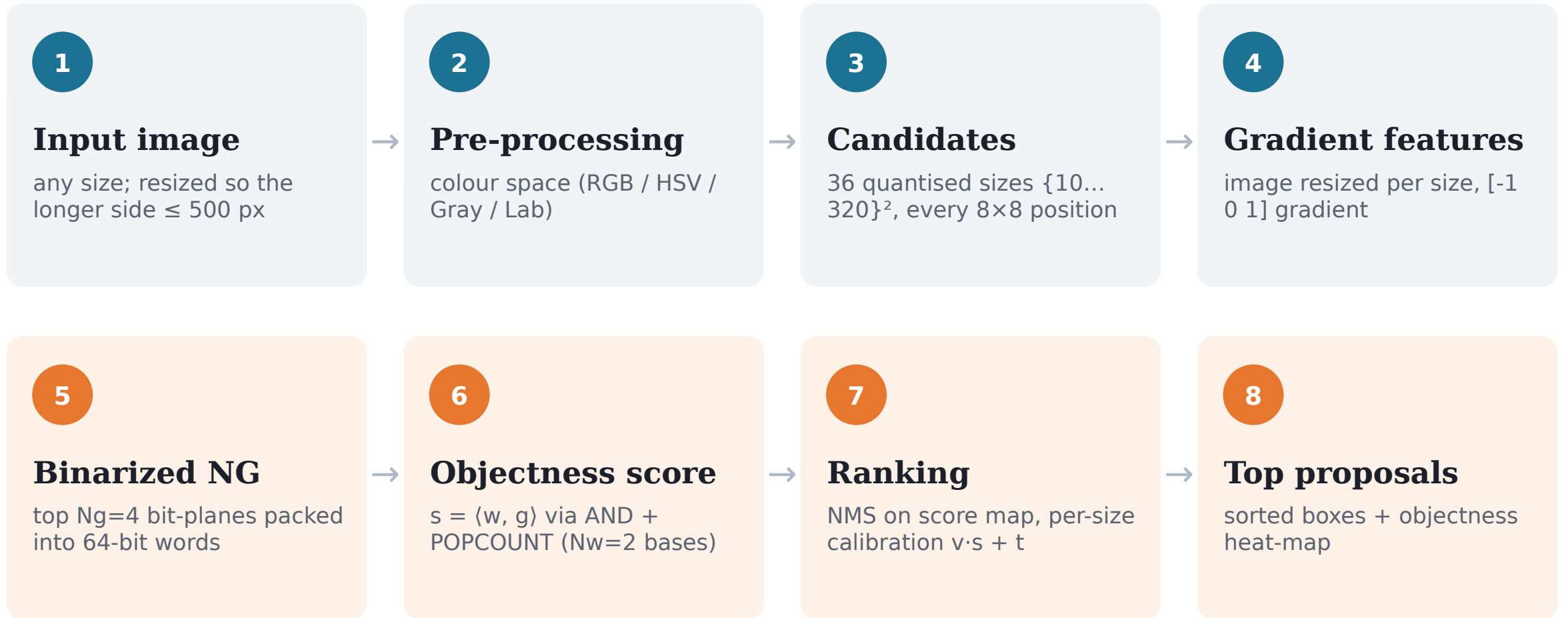
Binarize everything. w and g are approximated by binary vectors so a window score costs only a few BITWISE AND + POPCOUNT ops $\rightarrow$ 300 fps in the paper's C++.



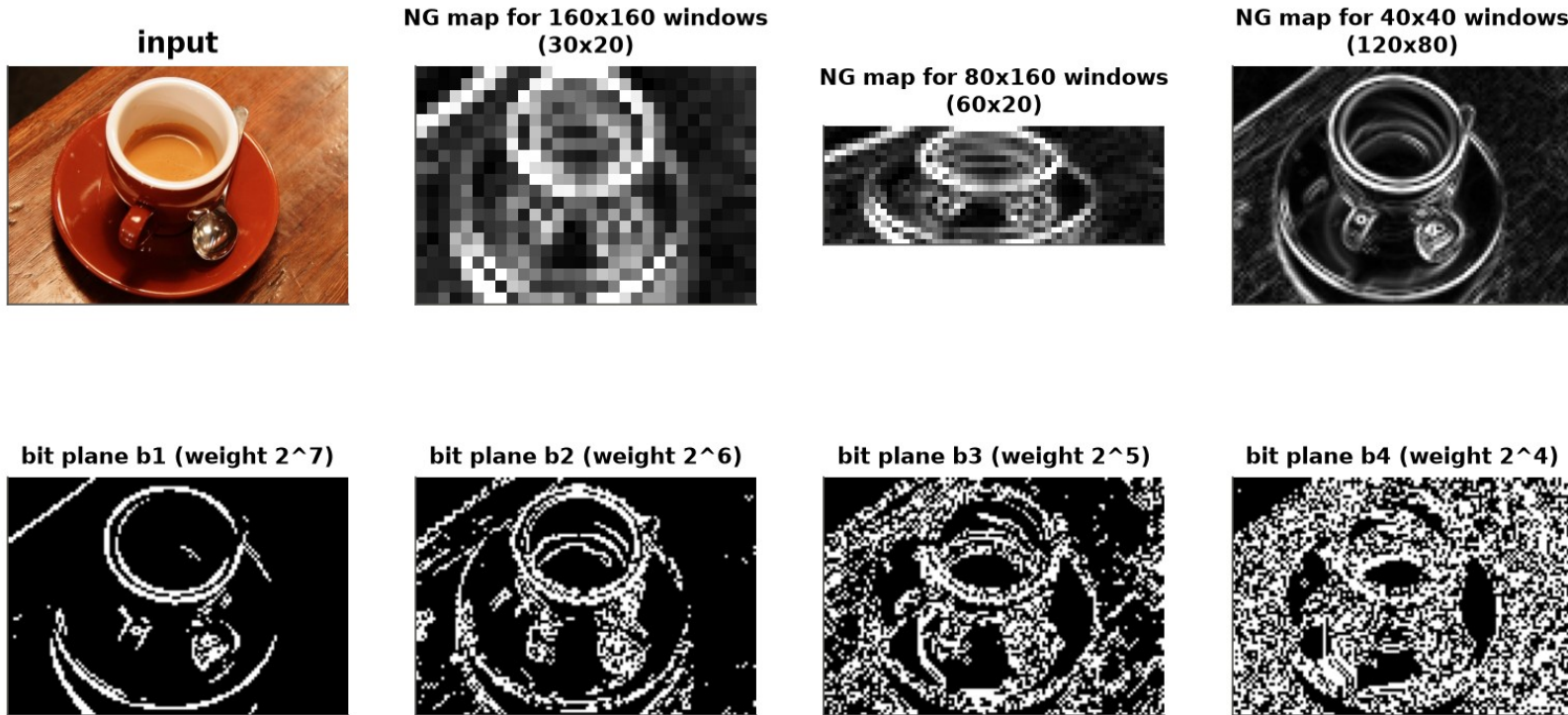
The learned 8×8 filter w (red = positive weight) and its binary approximations. It is a “closed boundary” template: positive on the border, negative inside.



Proposed system — 8 stages



Normed-gradient features at every scale



- The whole image is resized once per window size (w, h) to $(8W/w, 8H/h)$ — then every 8×8 patch of the NG map is one candidate window.
- Top row: NG maps for three window sizes. Bottom row: the four most significant bit-planes $b_1 \dots b_4$ that form the binarized feature.
- $g \approx \sum_k 2^{(8-k)} b_k$ — keeping $N_g = 4$ bits loses almost nothing (see ablation).

Sliding a 64-bit window: each new pixel updates a row byte with one shift + OR, and 8 row bytes form the 64-bit word (paper Alg. 2).

From dot product to bit operations

1 • Approximate the filter (Alg. 1)

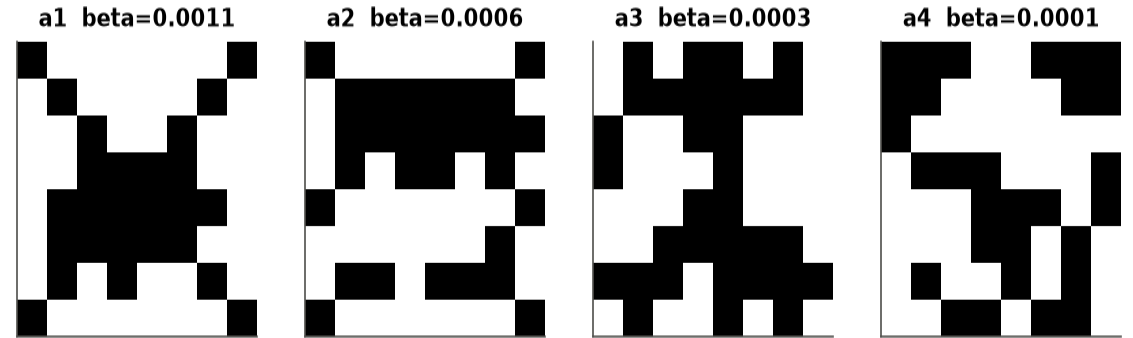
$w \approx \sum_j \beta_j a_j$, $a_j \in \{-1, +1\}^{64}$ (greedy: $a_j = \text{sign}(\text{residual})$)

2 • Approximate the feature

$g \approx \sum_k 2^{(8-k)} b_k$, $b_k \in \{0, 1\}^{64}$ (top N_g bit-planes)

3 • Score with bits only

$\langle a_j, b_k \rangle = 2 \cdot \text{popcount}(a_j^+ \text{ AND } b_k) - \text{popcount}(b_k)$
 $s \approx \sum_j \beta_j \sum_k 2^{(8-k)} \langle a_j, b_k \rangle$



- With $N_w = 2$ and $N_g = 4$ a window needs $4 \times (1 + 2) = 12$ POPCOUNTs instead of 64 multiply-adds.
- Accuracy is unchanged: DR@1000 95.1% (binary) vs 94.6% (float w) on the synthetic test set.
- Unit-tested: the bitwise score equals $\langle \tilde{w}, \tilde{g} \rangle$ computed with floats to 1e-3.

What we built

`bing/features.py`

NG maps, colour spaces, [-1 0 1] vs Sobel, per-box features

`bing/binary.py`

Alg. 1 filter binarization, 64-bit packing, AND+POPCOUNT scorer (numpy)

`bing/fast.py`

Numba JIT kernel of Alg. 2 — shift/OR feature packing, parallel popcount

`bing/model.py`

Two-stage training (LinearSVC + per-size calibration), NMS, ranking, heat-map

`bing/metrics.py`

DR-#WIN, MABO, AUC, recall-vs-IoU

`bing/datasets.py`

Synthetic scene generator, real photos, PASCAL VOC 2007 loader

Scripts

`make_synthetic.py` ·
`run_experiments.py` · `make_figures.py` ·
`demo.py` (image / webcam)

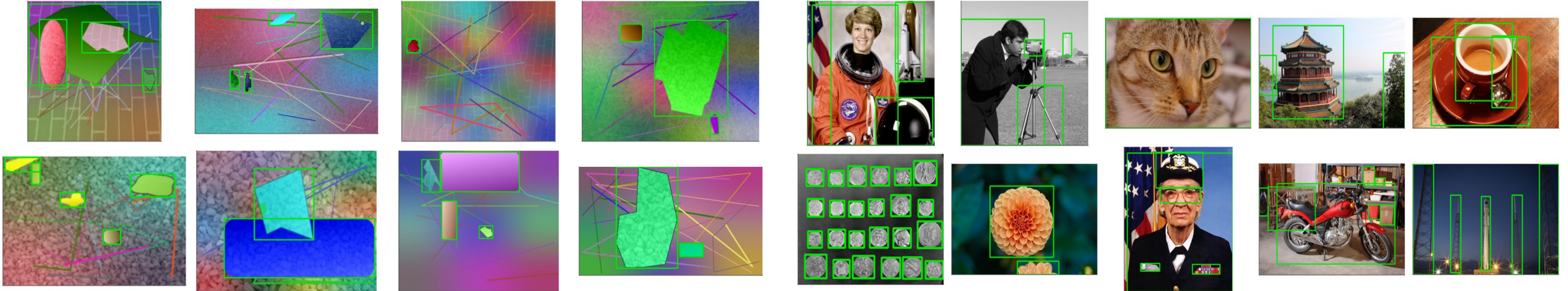
Stack

Python · OpenCV · NumPy · scikit-learn
· Numba · Matplotlib

Reference

Design follows `torrvision/Objectness` (C++); re-written from scratch in Python, no deep learning.

Evaluation data



Synthetic benchmark

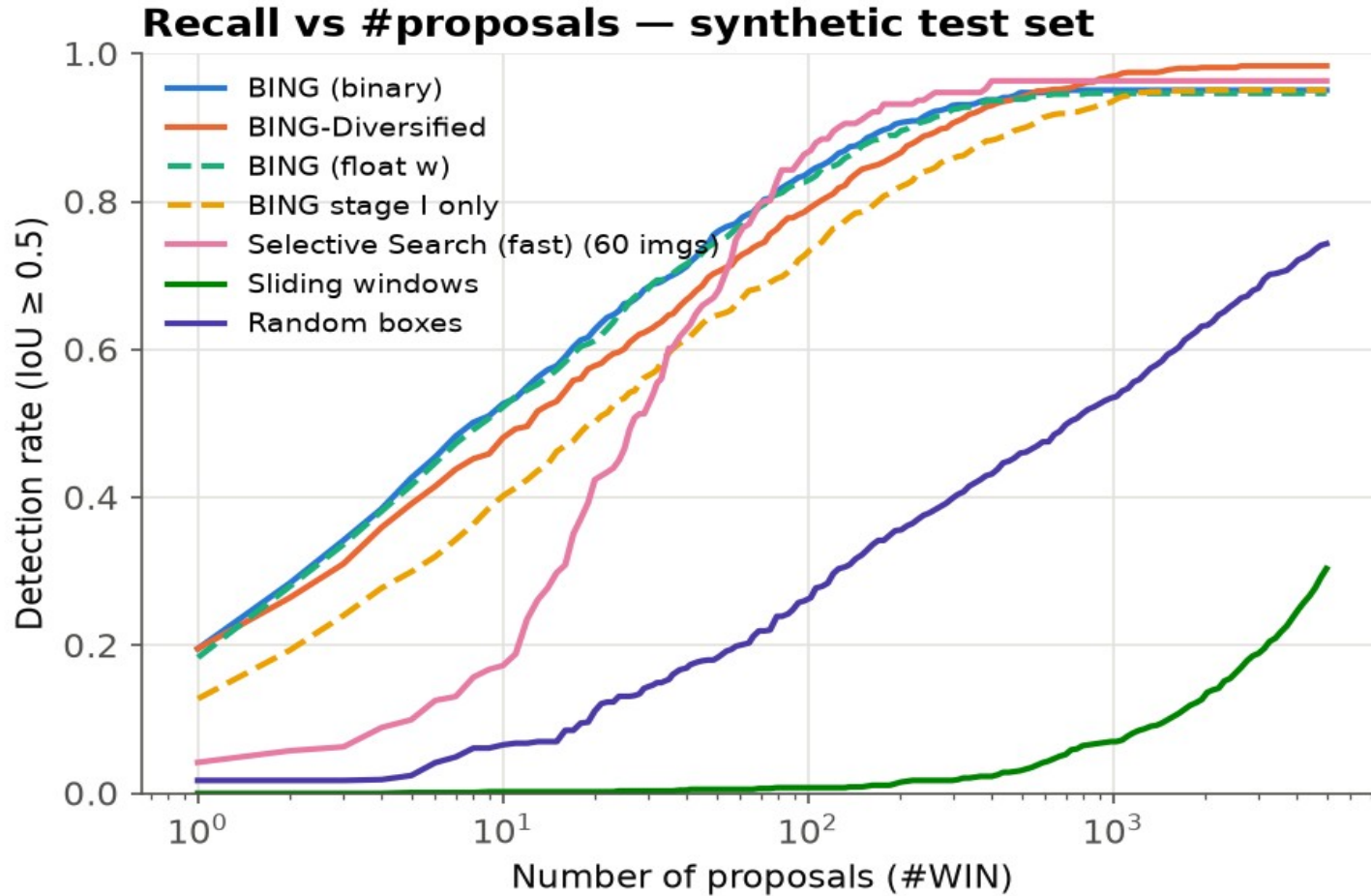
- 600 train / 300 test images, 1-5 objects each (915 test objects)
- Textured / smooth backgrounds + random clutter lines (edges that are not objects), shading, noise, blur

Real photographs

- 10 unseen natural images, 58 hand-labelled boxes (people, cup, coins, motorcycle, pagoda ...)
- Model trained only on synthetic data → tests generalisation of the 'closed boundary' cue

Metrics: detection rate (DR, $IoU \geq 0.5$) vs number of proposals, MABO, AUC, runtime. A VOC 2007 loader is included for the paper's full protocol.

Recall vs number of proposals



95.1%

DR@1000 — BING (binary)

97.0%

DR@1000 — BING-Diversified (RGB + HSV + Gray)

53.6%

DR@1000 — random boxes (no objectness)

How does BING compare?

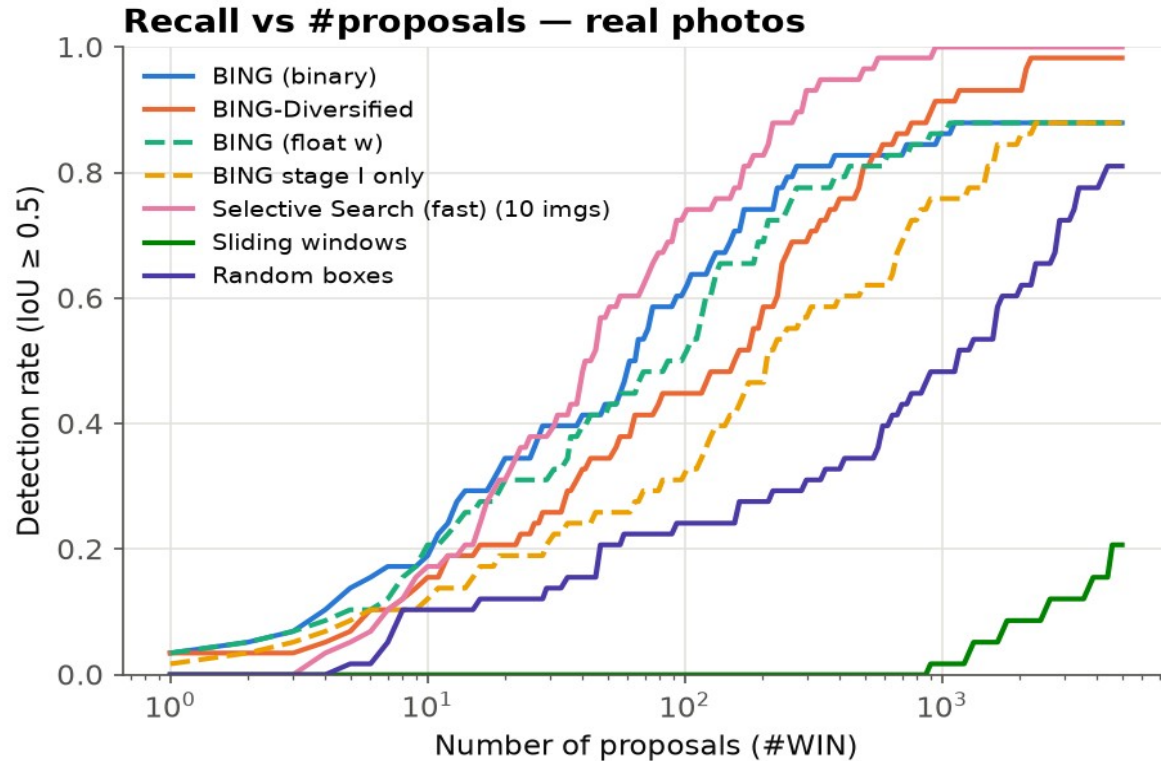
Method	DR@100	DR@1k	MABO@1k	AUC	img/s †
BING (binary)	84.0%	95.1%	0.658	0.825	30.3
BING (float w)	82.7%	94.6%	0.658	0.820	62.2
BING stage I only	73.2%	93.8%	0.653	0.758	22.0
BING-Diversified	79.0%	97.0%	0.675	0.808	20.2
Selective Search (fast) *	86.9%	96.3%	0.885	0.778	1.5
Random boxes	26.3%	53.6%	0.518	0.365	—
Sliding windows	0.8%	7.0%	0.314	0.052	10.7

* Selective Search on 60 test images (it is ~40× slower); BING on the same subset: DR@1000 97.4%. † End-to-end timing during evaluation (default INTER_AREA resizing, Python overhead) — see the Speed slide for the controlled benchmark.

Take-aways

- Stage II calibration matters: +11 pts DR@100.
- Binary $\approx$ float accuracy.
- Selective Search localises tighter (higher MABO) but is 40× slower.
- Exactly the trade-off the paper reports: BING is excellent at IoU 0.5, weaker at high IoU.

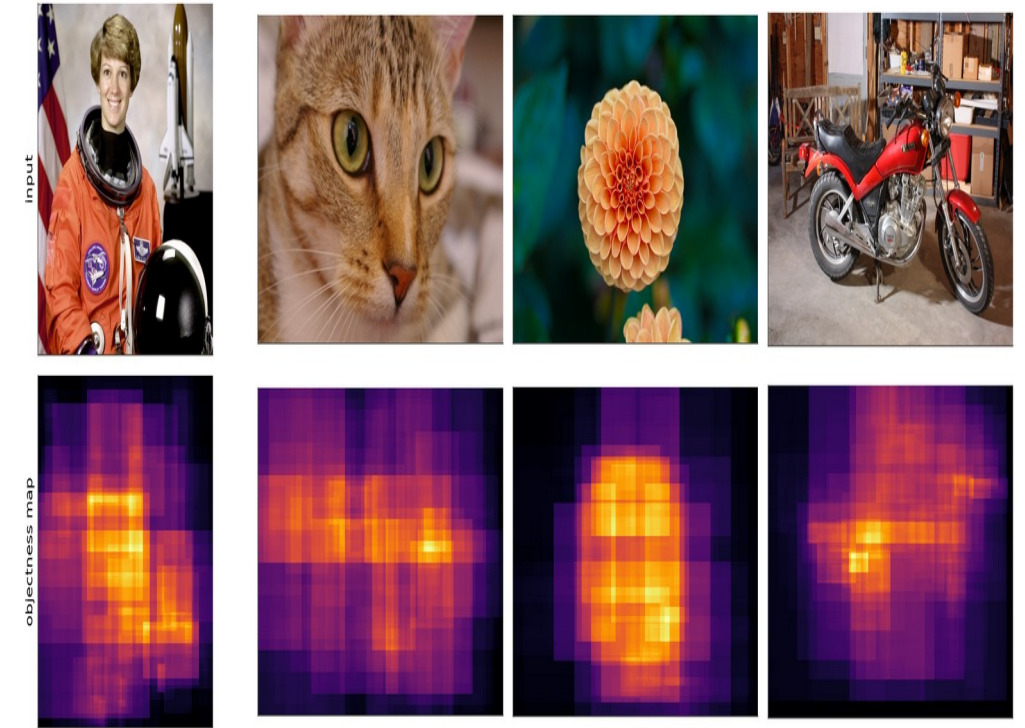
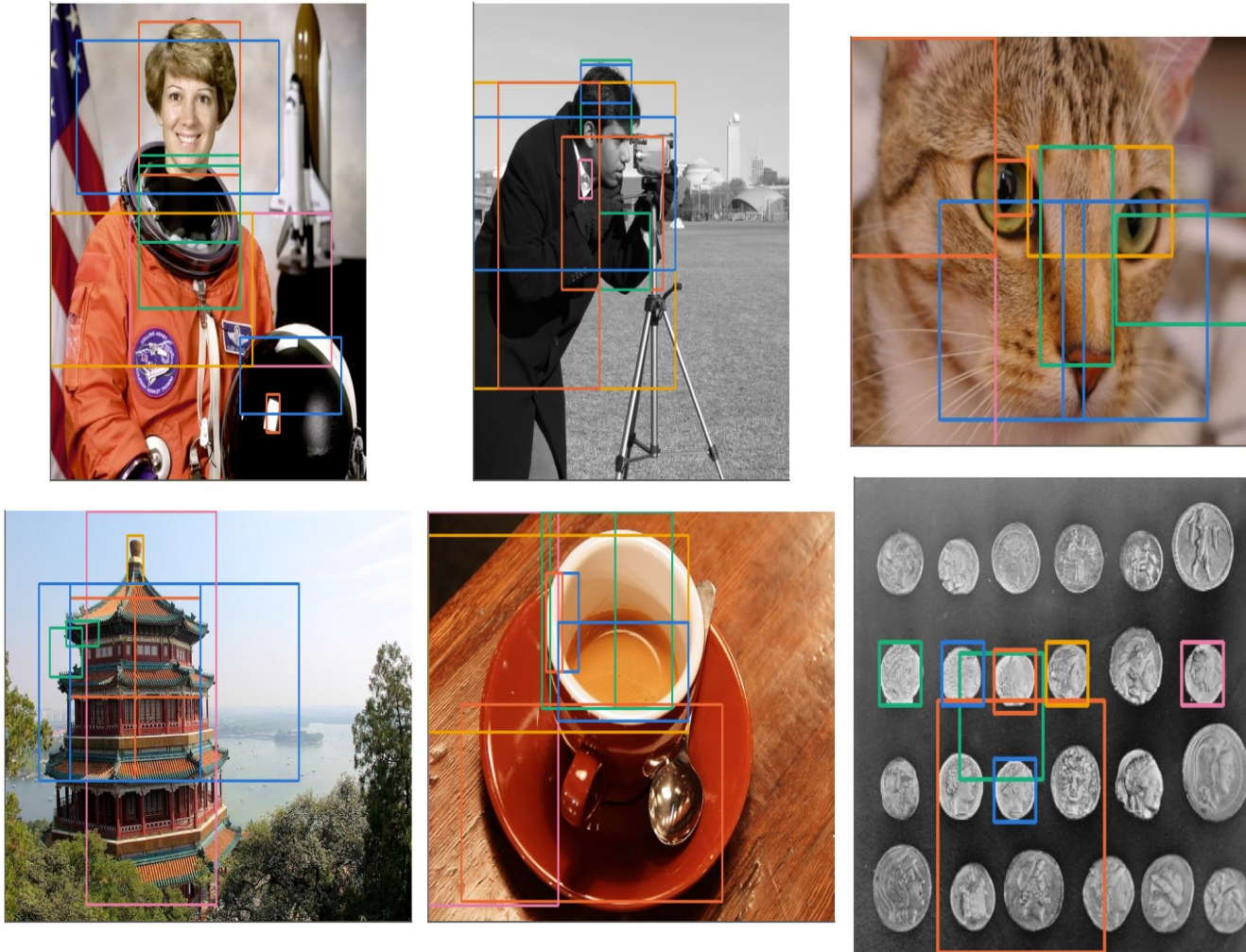
Generalisation to real photographs



Trained only on synthetic scenes, BING recovers 86.2% of 58 real objects in its top 1000 proposals (Diversified: 91.4%; random: 48.3%). Right: best proposal per object among the top 1000 — green = found ($\text{IoU} \geq 0.5$), red = missed.

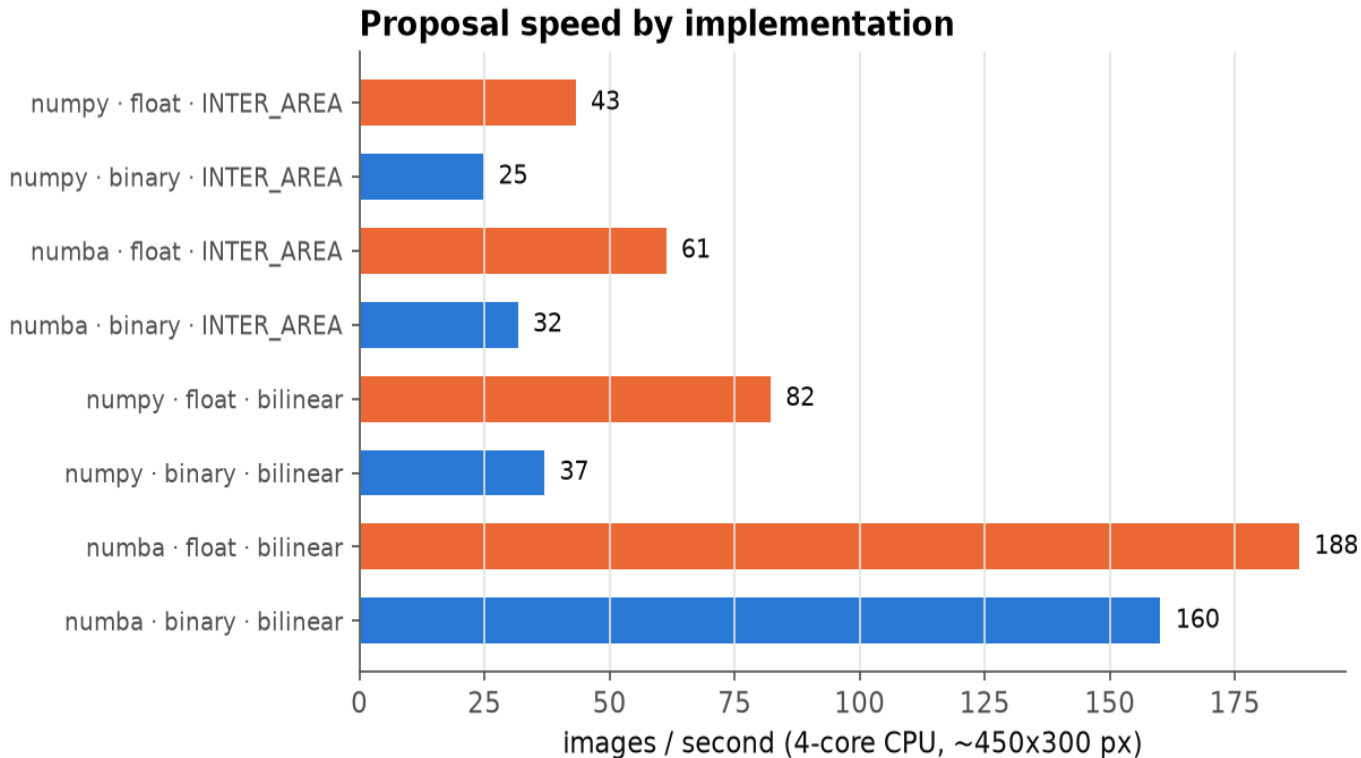


What BING looks at

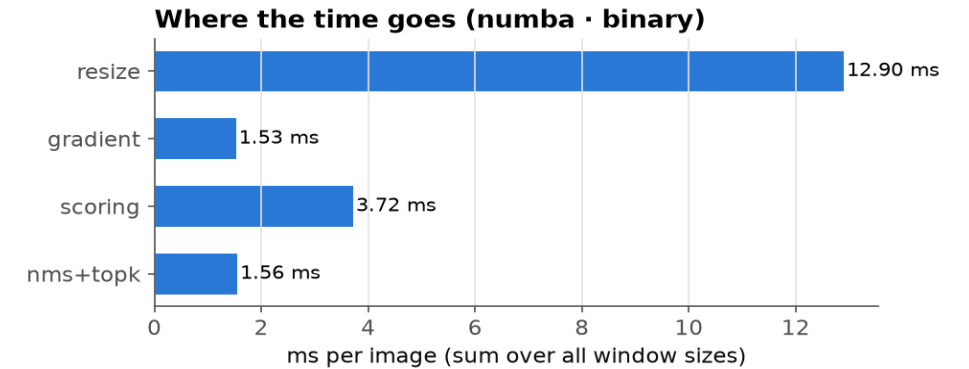


Left: the 8 highest-ranked proposals. Right: objectness heat-map (sum of the top 2000 proposal scores per pixel) — mass concentrates on the cup, the pagoda, the flower and the person rather than on texture.

Speed

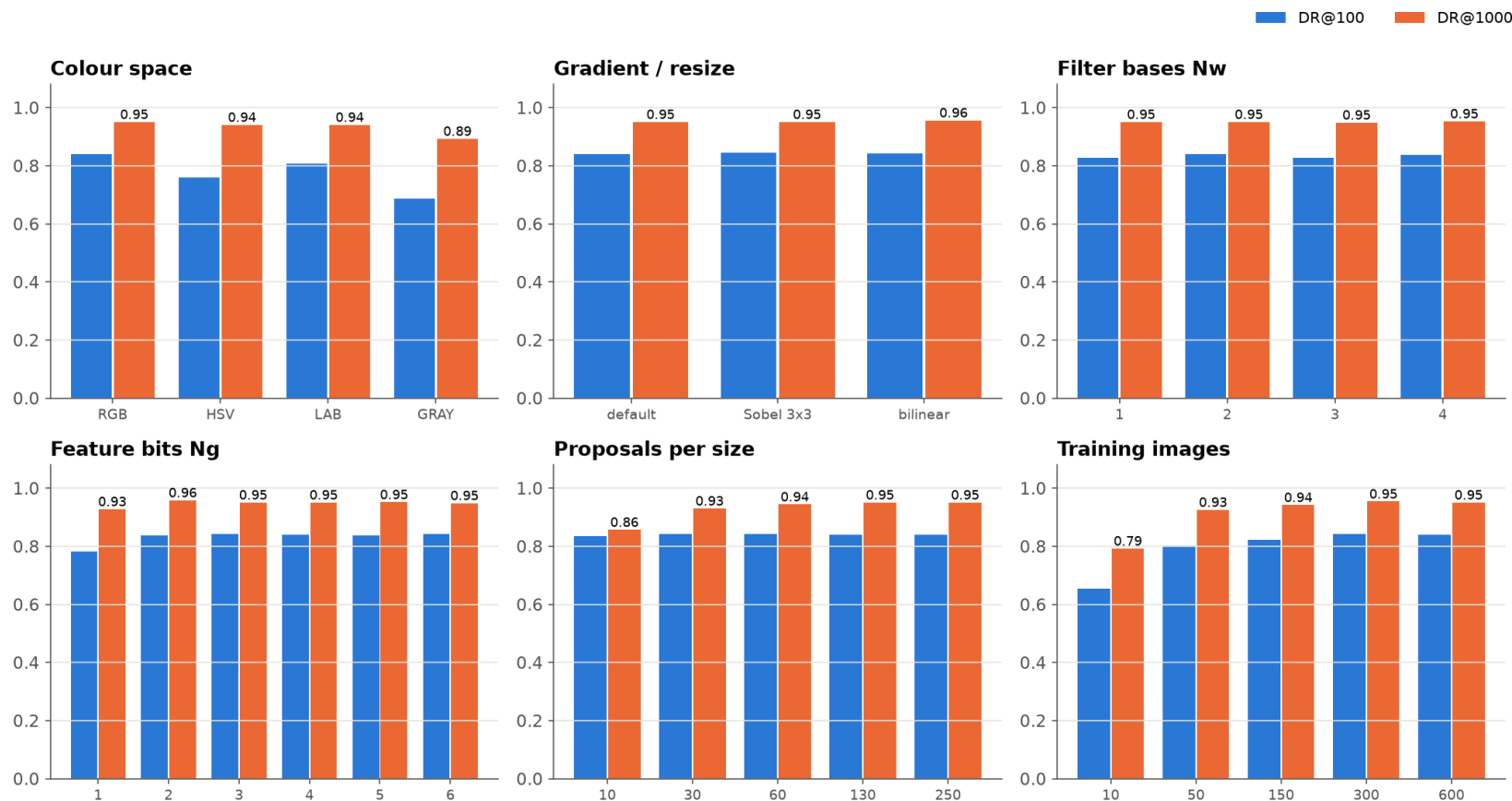


Blue = binarized scoring, orange = float filter. INTER_AREA gives anti-aliased resizing; bilinear matches the original C++ and is ~2× faster.



- Numba JIT of the bitwise kernel: 160 img/s vs 37 img/s in pure numpy.
- OpenCV's SIMD float correlation is still competitive in Python (188 img/s): the paper's 300 fps comes from hand-written C++ with SSE popcount.
- Per image, resizing dominates (12.9 ms with INTER_AREA) — scoring all windows with bit operations takes only 3.7 ms.
- Selective Search: ~1 img/s — BING is 1-2 orders of magnitude faster.

Ablation studies



Gradient representation: $[-1 \ 0 \ 1]$
95.1% vs Sobel 95.1% — the simplest mask is enough.

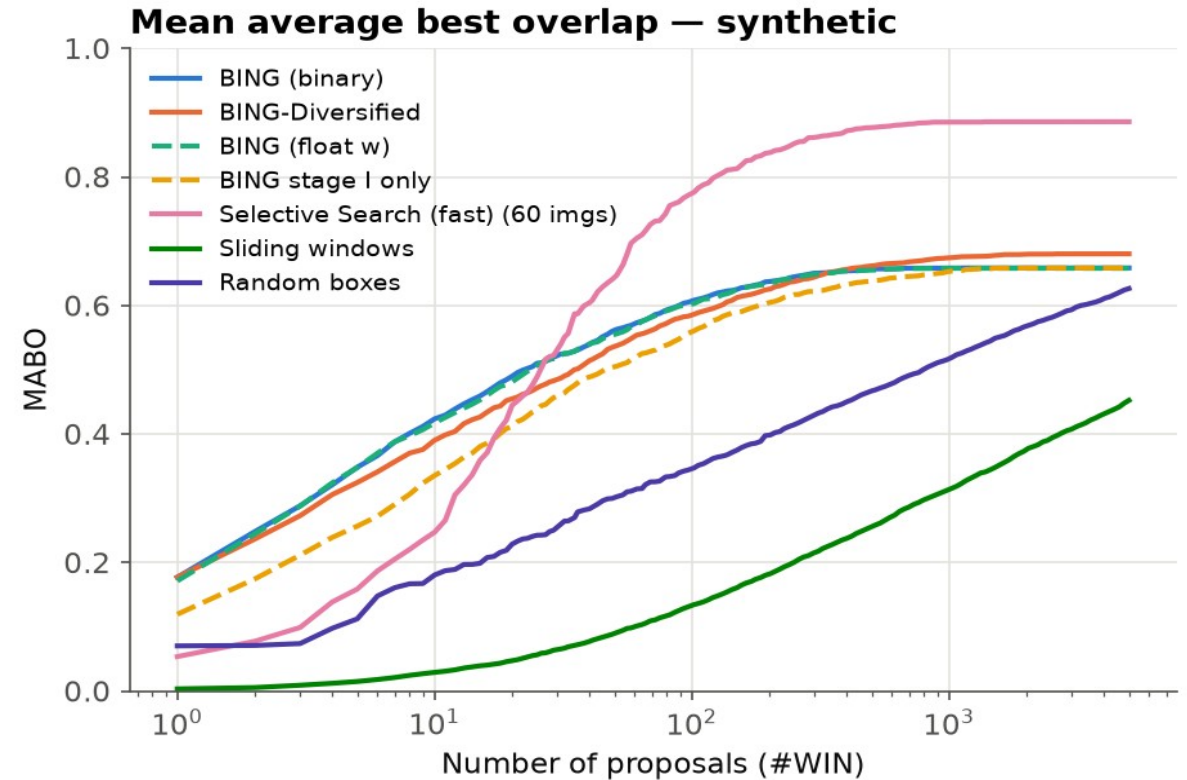
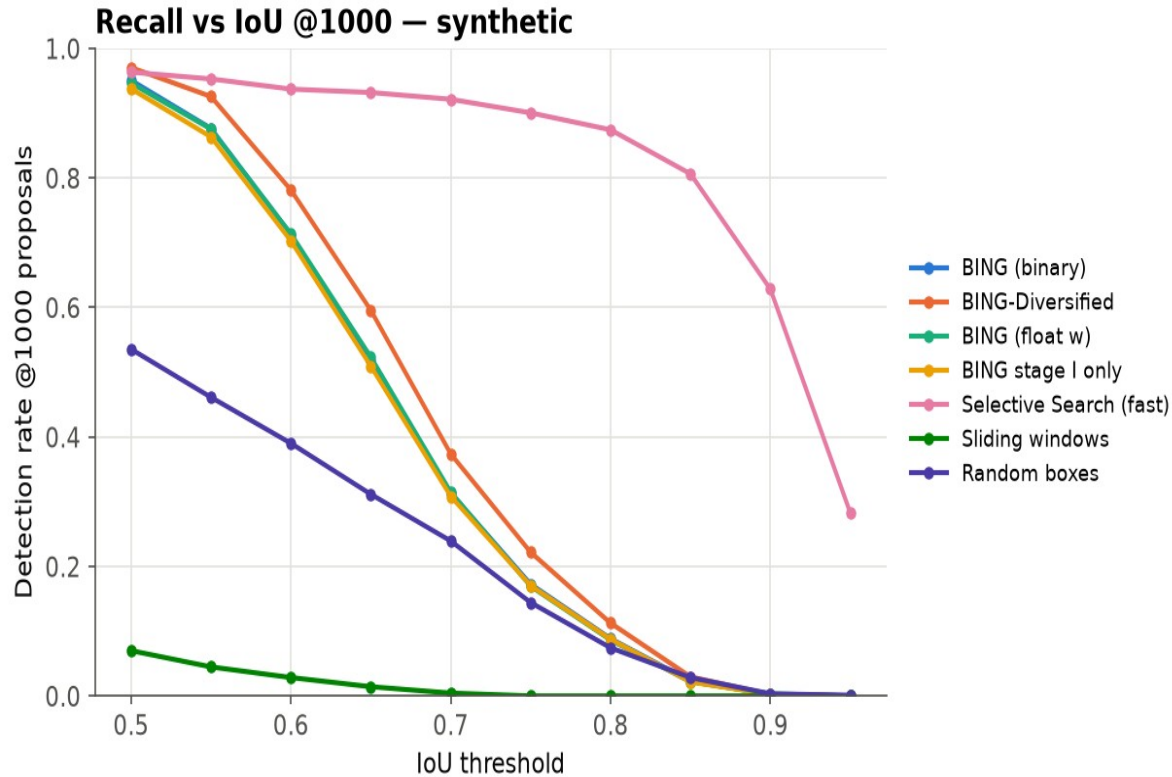
Colour space: RGB 95.1%, HSV 94.1%, Lab 94.1%, Gray 89.3%; they are complementary (Diversified).

Binarization: $Nw = 1 \rightarrow 95.1\%$, $Nw = 2 \rightarrow 95.1\%$; $Ng = 1 \rightarrow 92.8\%$, $Ng = 4 \rightarrow 95.1\%$.

Data: only 10 training images already give 79.2% — 64 weights are easy to learn.

All numbers: DR@1000 on the synthetic test set; bars also show DR@100.

Proposal quality vs localisation precision



BING's quantised window sizes (powers of two) and 8-pixel stride give coarse boxes: recall is high at IoU 0.5 but falls quickly as the threshold rises, and MABO saturates around 0.66. Segmentation-based Selective Search is slower but tighter. This matches known analyses of BING (Hosang et al., TPAMI 2016).

Planned contributions → delivered



Analysis of gradient representations

[-1 0 1] vs Sobel, 4 colour spaces, bit depth Ng



Proposal quality vs computational cost

binary vs float, numpy vs Numba, Nw/Ng trade-offs, BING vs Selective Search



Different numbers of proposals

DR / MABO curves from 1 to 5000 proposals, per-size budget ablation



Visualisation of objectness scores

learned filter, bit-planes, calibration map, heat-maps, top-k boxes



Different image categories / datasets

synthetic benchmark, 10 real photos, VOC 2007 loader



Optimisation / modernisation

pure-Python package + Numba Alg. 2 kernel, unit tests, webcam demo

Limitations & future work

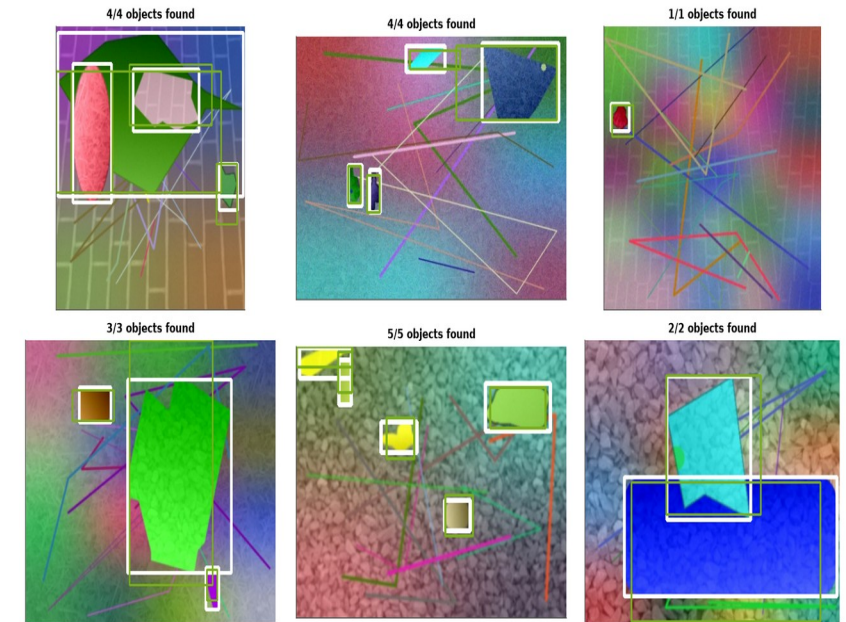
Localisation is coarse. Power-of-two sizes and an 8-px stride cap MABO; refine top boxes (e.g. edge-based box regression) or add intermediate sizes.

Evaluation scale. PASCAL VOC could not be downloaded in our sandbox; we provide the loader and scripts to reproduce the paper's protocol (train on trainval, test on test).

Python ceiling. Numba closes much of the gap, but a SIMD C++/Cython popcount kernel would be needed for the paper's 300 fps.

Synthetic-to-real gap. Training on real annotated images (VOC / COCO) would raise real-photo recall further.

Next step. Feed BING proposals into a classifier (HOG + SVM or a small CNN) for a complete detector.



Misses (red) are mostly low-contrast objects or objects cut by clutter lines.

CONCLUSION

Simple gradients, a 64-weight linear model and bit operations are enough to find most objects — fast.

95.1%

objects found in top 1000
(synthetic test)

86.2%

objects found in top 1000
(real photos, zero real
training)

188

images / second
(Python + Numba, 4 CPU
cores)

12

POPCOUNTs per window
instead of 64 multiply-adds

References [1] M.-M. Cheng, Z. Zhang, W.-Y. Lin, P. H. S. Torr, “BING: Binarized Normed Gradients for Objectness Estimation at 300fps,” CVPR 2014. [2] J. Uijlings et al., “Selective Search for Object Recognition,” IJCV 2013. [3] B. Alexe, T. Deselaers, V. Ferrari, “Measuring the Objectness of Image Windows,” TPAMI 2012. [4] J. Hosang et al., “What makes for effective detection proposals?,” TPAMI 2016. [5] P. Arbeláez et al., “Multiscale Combinatorial Grouping,” CVPR 2014.

Thank you — questions?